

API Documentation

1. Scope

This repository contains two integration surfaces:

1. An external HTTP score-code endpoint used by `Games/Tetris/NetSubmit.h`.
2. A serial host-runtime packet protocol used by `Games/Tetris/HostRuntime.cpp` and `Games/Tetris/Tetris.ino`.

No Swagger/OpenAPI file is present.

2. HTTP API: score code creation

2.1 Endpoint summary

Method	Route	Implemented where	Purpose
POST	<code>/api/codes</code>	Client call in <code>Games/Tetris/NetSubmit.h</code>	Submit a signed Tetris score and receive a one-time display code.

The route is joined with `API_BASE`, which is expected to come from `NetConfig.h`.

2.2 `POST /api/codes`

Purpose

Submit a completed Tetris score to an external service and obtain a code for display on the PixelGrid segment panel.

Request headers

Header	Value
<code>Content-Type</code>	<code>application/json</code>

Request body

The embedded client constructs this JSON body:

```
{
  "score": 1234,
  "game_code": "TETRIS_OR_CONFIGURED_CODE",
  "ts": 1700000000,
  "nonce": "hexadecimal_nonce",
  "sig": "base64url_hmac_sha256_signature"
}
```

Request fields

Field	Type	Required	Source	Validation evidenced in client
<code>score</code>	integer	Yes	Game-over score from <code>TetrisGame::score</code>	Cast to <code>int</code> ; no explicit range validation in client.
<code>game_code</code>	string	Yes	<code>GAME_CODE</code> from <code>NetConfig.h</code>	No client validation beyond inclusion in canonical message.
<code>ts</code>	integer Unix timestamp	Yes	<code>time(nullptr)</code> after NTP sync	Client refuses to submit if timestamp is below <code>1700000000</code> . A source comment states that the server enforces a 120-second time window.
<code>nonce</code>	string	Yes	Two <code>esp_random()</code> values formatted as hexadecimal	Generated per submission; no client-side uniqueness store.
<code>sig</code>	string	Yes	HMAC-SHA256 of canonical message using <code>GAME_SECRET</code>	Empty signatures abort the request.

Signature algorithm

The canonical string is constructed in this exact field order:

```
game_code=<game_code>&score=<score>&ts=<ts>&nonce=<nonce>
```

The signature is HMAC-SHA256 using `GAME_SECRET`, encoded as base64url without trailing padding characters.

Response structure

The client expects a JSON object containing a string field named `code`:

```
{
  "code": "ABC123"
}
```

Response handling

- Any HTTP status code outside the 2xx range is treated as failure.
- A missing `code` string is treated as failure.
- On success, the returned code is assigned to the output string and later displayed on the segment panel by the Tetris game-over flow.

Possible error responses

The client is prepared for these error classes:

Error class	Client behaviour
Wi-Fi connection failure	Returns <code>false</code> ; no HTTP request is made.
NTP time failure	Returns <code>false</code> ; no signed request is made.
HMAC/signature failure	Returns <code>false</code> ; no HTTP request is made.
HTTP client begin failure	Returns <code>false</code> .
Non-2xx HTTP response	Logs response body to serial and returns <code>false</code> .
Missing <code>code</code> in response	Logs diagnostic output and returns <code>false</code> .

Evidence gaps

- The server-side controller, validation rules, status codes, and response schema are not in the repository.
- No OpenAPI/Swagger specification exists.
- `NetConfig.h` is not present, so exact `API_BASE`, `GAME_CODE`, and secret names are inferred only from client references.

3. Serial host protocol

Tetris also exposes a serial packet interface for host-controlled display updates and input feedback. This is not a network API, but it is an integration contract.

3.1 Device-to-host input packet

Direction	Marker	Payload	Purpose
Device to host	ASCII <code>b</code>	One packed input byte	Notify a host application when button/joystick state changes.

Packed input byte

`Games/Tetris/Tetris.ino` packs stable input states into bits:

Bit	Meaning
0	Button 1 stable pressed
1	Button 2 stable pressed
2	Button 3 stable pressed
3	Joystick up stable pressed
4	Joystick down stable pressed
5	Joystick left stable pressed

Bit	Meaning
6	Joystick right stable pressed
7	Button 4 stable pressed

The sender throttles updates using `HOST_SEND_MIN_MS` and only sends when the packed payload changes.

3.2 Host-to-device LED frame packet

Direction	Header	Length field	Payload	Purpose
Host to device	PBFR	2-byte little-endian length	GRB bytes for the LED frame	Replace the matrix frame from a host-rendered source.

Validation rules

- Header must begin with **P** and then **BFR**.
- Payload length must match the expected `hostGrb` buffer size.
- Invalid lengths are ignored and parsing resumes.

3.3 Host-to-device LCD text packet

Direction	Header	Length field	Payload	Purpose
Host to device	PBLC	2-byte little-endian length	UTF-8/ASCII text payload	Update the segment-panel text display.

Validation rules

- Payload is capped to a safe local buffer before display.
- Excess bytes are flushed so the parser remains synchronised.

3.4 Host-to-device HUD segment packet

Direction	Header	Length field	Payload	Purpose
Host to device	PB7S	2-byte little-endian length	15 bytes	Update three HUD masks, colours, and score digits.

Payload structure

Byte range	Meaning
0..2	Segment masks for digits 0..2.
3..11	RGB colour triplets for digits 0..2.
12..14	Score digits as hundreds, tens, and ones.

Validation rules

- Payload length must be exactly 15 bytes.
- If the length is not 15, the payload is flushed and ignored.

4. Internal function-level interfaces

Interface	Location	Purpose
<code>TetrisGame::update</code> and <code>TetrisGame::render</code>	<code>Games/Tetris/Game.h</code>	Apply input/time to game state and render the board.
<code>Input::update</code> , <code>Input::latch</code> , <code>Input::state</code> , <code>Input::horizontalRepeat</code>	<code>Games/Tetris/Input.h</code>	Convert raw pins to stable gameplay input.
<code>Renderer</code> methods	<code>Games/Tetris/Render.h</code>	Draw matrix cells, text, HUD masks, and game-over/title displays.
<code>Game_reset</code> , <code>Game_stepBallOnce</code> , <code>Game_movePaddle</code>	<code>Games/Breakout/Game.h</code> and <code>.cpp</code>	Breakout gameplay interface used by <code>Breakout.ino</code> .
<code>Render_begin</code> , <code>Render_renderFrame</code>	<code>Games/Breakout/Render.h</code> and <code>.cpp</code>	Breakout display lifecycle.